MINI PROJECT –02

TITLE-

CI/CD Pipeline: Fast Food Application Deployment using Jenkins and AWS Code-Pipeline.

Project Objective-

To design and implement a complete **CI/CD pipeline** for a Fast Food web application using:

- Jenkins (for Continuous Integration)
- AWS CodePipeline (for Continuous Delivery)
- AWS services like EC2, S3, CodeBuild, CodeDeploy

The goal is to automate build, test, and deployment processes.

Project Flow –

Step 1: Create EC2 Instance

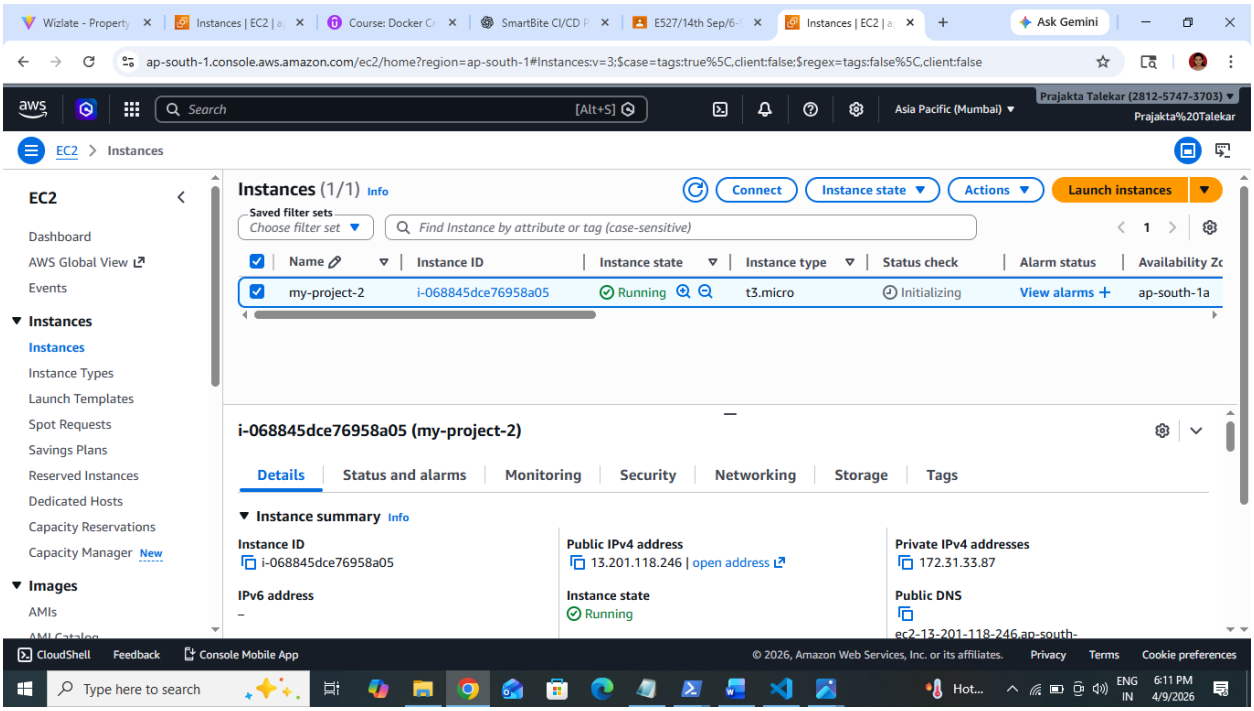
First, create an EC2 instance and assign a proper name for easy identification.
While creating the instance, generate a key pair, which is required for

secure access to the server using SSH. It ensures that only authorized users can connect to the instance.

Next, select Ubuntu as the operating system.
Configure the inbound security group rules to allow required ports, such as:

- Port 22 (SSH) for remote access
- Port 8080 for Jenkins
- Port 80 (HTTP) for the application

After that, increase the storage capacity based on project requirements. Finally, review all the configurations and click on Launch Instance to create the EC2 instance.



Step 2: Connect to EC2 Instance - After launching the instance, connect to it using SSH from your terminal .This step allows you to access the server and install required tools like Jenkins and Git.

Step 3: Install Required Software

Install the following tools on the EC2 instance:

- Java (required for Jenkins)
- Jenkins
- Git
- Node.js

Why?

These tools are necessary to run Jenkins and manage the application build process.

Commands used are as follow-

1. Update package list from Ubuntu repositories

```
sudo apt update
```

2. Upgrade all installed packages to latest versions (non-interactive)

```
sudo apt upgrade -y
```

3. Download and add NodeSource repository for Node.js 20.x

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

4. Install Node.js (includes npm)

```
sudo apt install -y nodejs
```

5. Verify Node.js installation and check version

```
node --version
```

6. Verify npm installation and check version

```
npm --version
```

7. Install PM2 globally (production process manager for Node.js)

```
sudo npm install -g pm2
```

8. Verify PM2 installation and check version

```
pm2 --version
```

9. Generate startup script for PM2 to run on system boot

```
pm2 startup
```

10. Execute the PM2 startup command (COPY YOUR OWN GENERATED COMMAND)

This command sets up PM2 to start automatically on server reboot

```
sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2 startup systemd -u ubuntu --hp /home/ubuntu
```

11. Save current PM2 process list for resurrection on reboot

```
pm2 save
```

12. Check PM2 systemd service status

```
sudo systemctl status pm2-ubuntu
```

13. Install Java 17 JRE (required for Jenkins)

```
sudo apt install -y fontconfig openjdk-17-jre
```

14. Verify Java installation and check version

```
java --version
```

15. Remove old Jenkins repository list if exists (cleanup)

```
sudo rm -f /etc/apt/sources.list.d/jenkins.list
```

16. Remove old Jenkins GPG keys if exist (cleanup)

```
sudo rm -f /usr/share/keyrings/jenkins*
```

17. Download Jenkins GPG key and add to system keyring

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key |
sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

18. Add Jenkins stable repository to apt sources

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee
/etc/apt/sources.list.d/jenkins.list
```

19. Update package list again to include Jenkins repository

```
sudo apt update
```

20. Install Jenkins

```
sudo apt install jenkins -y
```

21. Start Jenkins service

```
sudo systemctl start jenkins
```

22. Enable Jenkins to start automatically on system boot

```
sudo systemctl enable jenkins
```

23. Check Jenkins service status

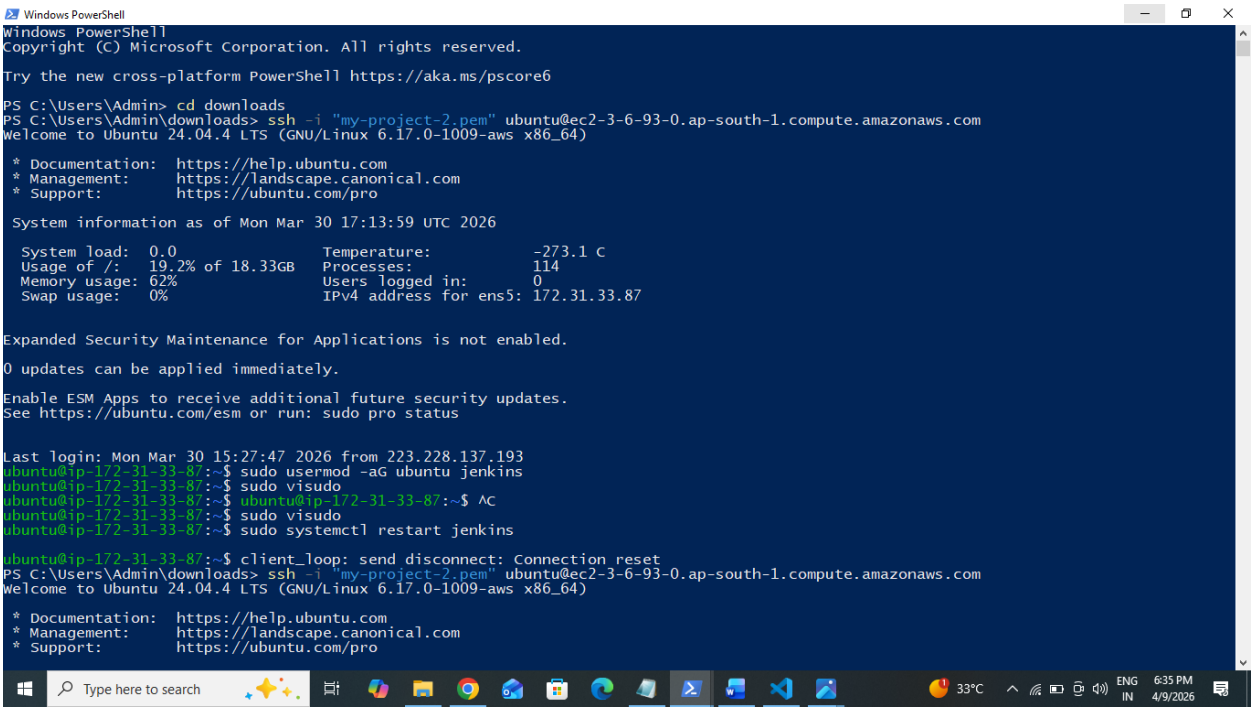
```
sudo systemctl status jenkins
```

24. Display Jenkins initial admin password (needed for first login)

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

25. Install Ruby and wget (dependencies for CodeDeploy agent)

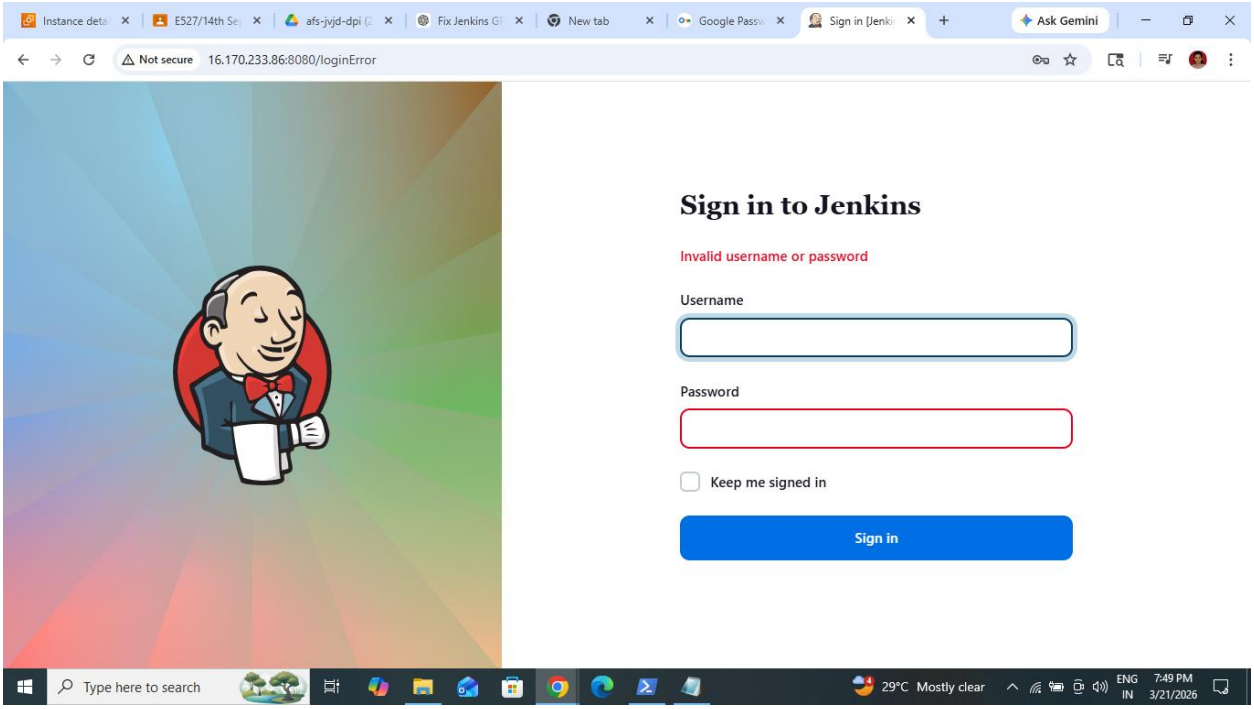
```
sudo apt install -y ruby wget
# 26. Change directory to /tmp for temporary downloads
cd /tmp
# 27. Download AWS CodeDeploy agent installer script
wget https://aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com/latest/install
# 28. Make the installer script executable
chmod +x ./install
# 29. Install CodeDeploy agent automatically
sudo ./install auto
# 30. Start CodeDeploy agent service (legacy command)
sudo service codedeploy-agent start
# 31. Check CodeDeploy agent status (legacy command)
sudo service codedeploy-agent status
# 32. Start CodeDeploy agent using systemd
sudo systemctl start codedeploy-agent
# 33. Enable CodeDeploy agent to start on boot
sudo systemctl enable codedeploy-agent
# 34. Check CodeDeploy agent status using systemd
sudo systemctl status codedeploy-agent
```



Step 4: Configure Jenkins

- Access Jenkins using: http://<EC2-Public-IP>:8080
- Unlock Jenkins using the initial admin password
- Install suggested plugins
- Create an admin user

Jenkins is used to automate the build and integration process.



Install plugins as follow-

1. Node js
2. Git hub integration plugin
3. Pipeline stageview plugin

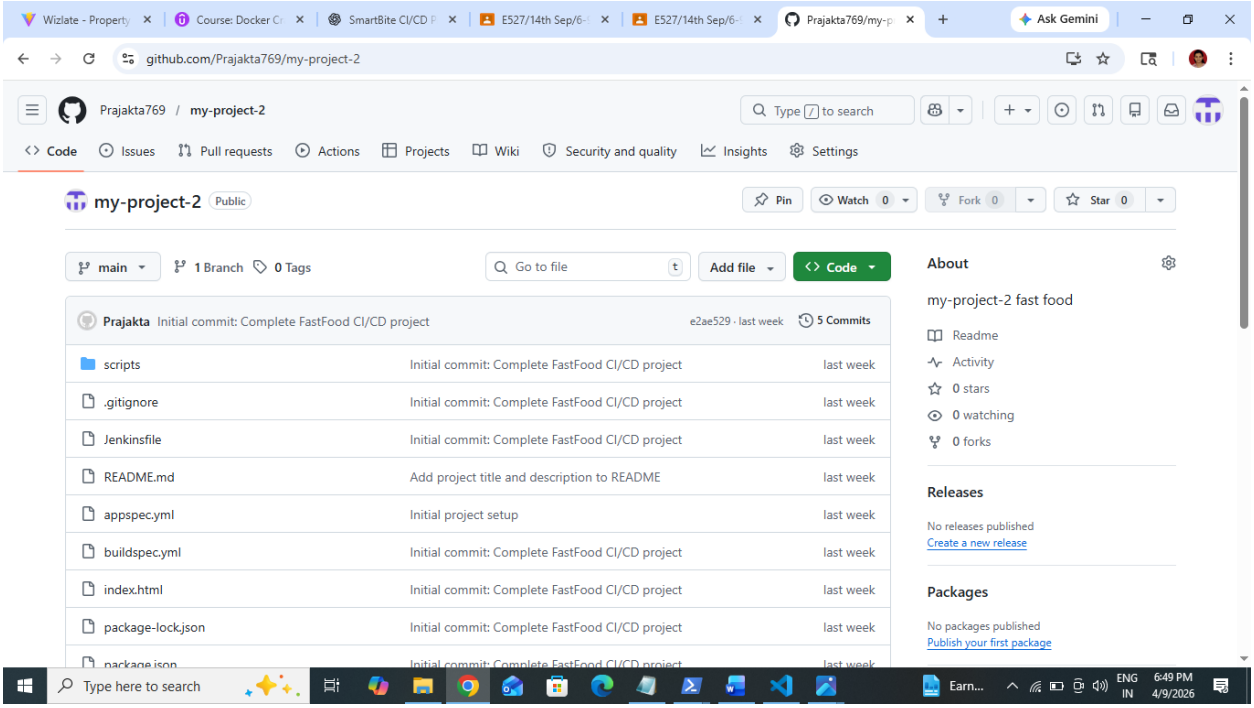
   Tools – nodejs-20

Step 5: Create GitHub Repository

Create a repository and upload your Fast Food application code.

Why?

GitHub acts as a centralized source code repository for the CI/CD pipeline.



USE FOLLOWING COMMANDS-

Navigate to downloads directory

cd downloads

Clone your FastFood CI/CD project repository

git clone https://github.com/Prajakta769/my-project-2.git

Change into project directory

cd fastfood-cicd-project

Step 6: Create Project Files in VS Code-

After creating the GitHub repository, open the project in **Visual Studio Code** and create the required project files and folder structure.

The following files are created to build and deploy the Fast Food application:

- **README.md** – to describe the project

- **Jenkinsfile** – to define CI/CD pipeline in Jenkins
- **package.json & package-lock.json** – for managing Node.js dependencies
- **server.js** – main backend application file
- **index.html** – main frontend page
- **style.css** – for styling the application
- **script.js** – for client-side functionality
- **appspec.yml** – configuration file for CodeDeploy
- **buildspec.yml** – configuration file for CodeBuild

  Also, create a **scripts folder** and add the following deployment scripts:
- install_dependencies.sh
- start_application.sh
- stop_application.sh
- validate_service.sh

  Additionally, create a **.gitignore file** to exclude unnecessary files like node_modules.

  Why?

  This step is important to:
- Organize the project structure properly
- Define CI/CD pipeline configuration
- Enable automated build and deployment using AWS services

Use following commands-

Create README.md file

```
New-Item -ItemType File -Path "README.md"\
```

Create Jenkinsfile for CI/CD pipeline

```
New-Item -ItemType File -Path "Jenkinsfile"
```

Create package.json for Node.js dependencies

```
New-Item -ItemType File -Path "package.json"
```

Create package-lock.json for dependency locking

```
New-Item -ItemType File -Path "package-lock.json"
```

Create server.js (main Node.js application file)

```
New-Item -ItemType File -Path "server.js"
```

Create index.html (main HTML page)

```
New-Item -ItemType File -Path "index.html"
```

Create style.css (CSS styling file)

```
New-Item -ItemType File -Path "style.css"
```

Create script.js (client-side JavaScript)

```
New-Item -ItemType File -Path "script.js"
```

Create appspec.yml (CodeDeploy deployment configuration)

```
New-Item -ItemType File -Path "appspec.yml"
```

Create buildspec.yml (CodeBuild build specification)

```
New-Item -ItemType File -Path "buildspec.yml"
```

Create scripts directory for deployment scripts

```
New-Item -ItemType Directory -Path "scripts"
```

Create install_dependencies.sh script

```
New-Item -ItemType File -Path "scripts\install_dependencies.sh"
```

Create start_application.sh script

```
New-Item -ItemType File -Path "scripts\start_application.sh"
```

Create stop_application.sh script

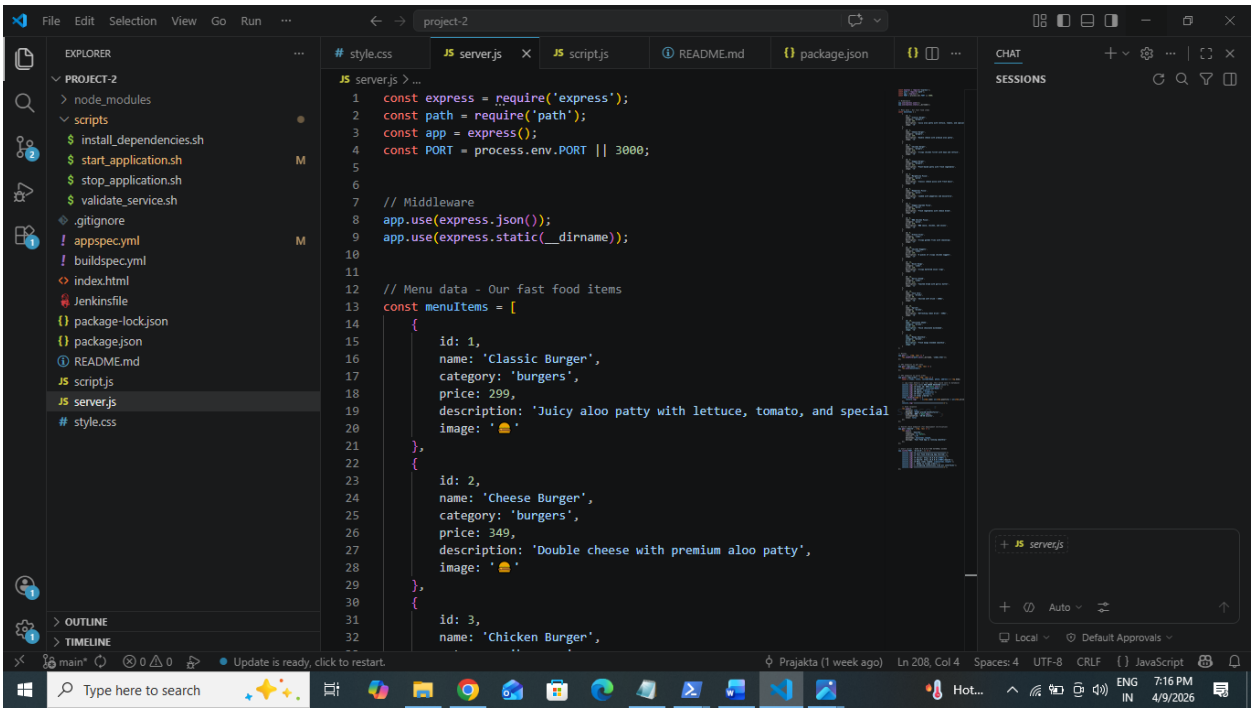
```
New-Item -ItemType File -Path "scripts\stop_application.sh"
```

Create validate_service.sh script

```
New-Item -ItemType File -Path "scripts\validate_service.sh"\
```

Create .gitignore file (to exclude node_modules, etc.)

```
New-Item -ItemType File -Path ".gitignore"
```



Open terminal in vs and add following commands

Stage all files for commit

```
git add .
```

Commit changes with message

```
git commit -m "Initial commit: Complete my-project-2"
```

Remove existing remote origin (if any)

git remote remove origin

Add new remote origin (your GitHub repository)

git remote add origin https://github.com/Prajakta769/my-project-2.git

Verify remote repository configuration

git remote -v

Push changes to GitHub main branch

git push -u origin main

Install npm dependencies for the project

npm install

Step 7: Create Jenkins Job / Pipeline

- Create a new job in Jenkins
- Configure source code (GitHub repo)
- Add build steps

Why?

This automates the **Continuous Integration (CI)** process.

Before that run these commands in terminal

1) Add Jenkins user to ubuntu group (allows Jenkins to access files)

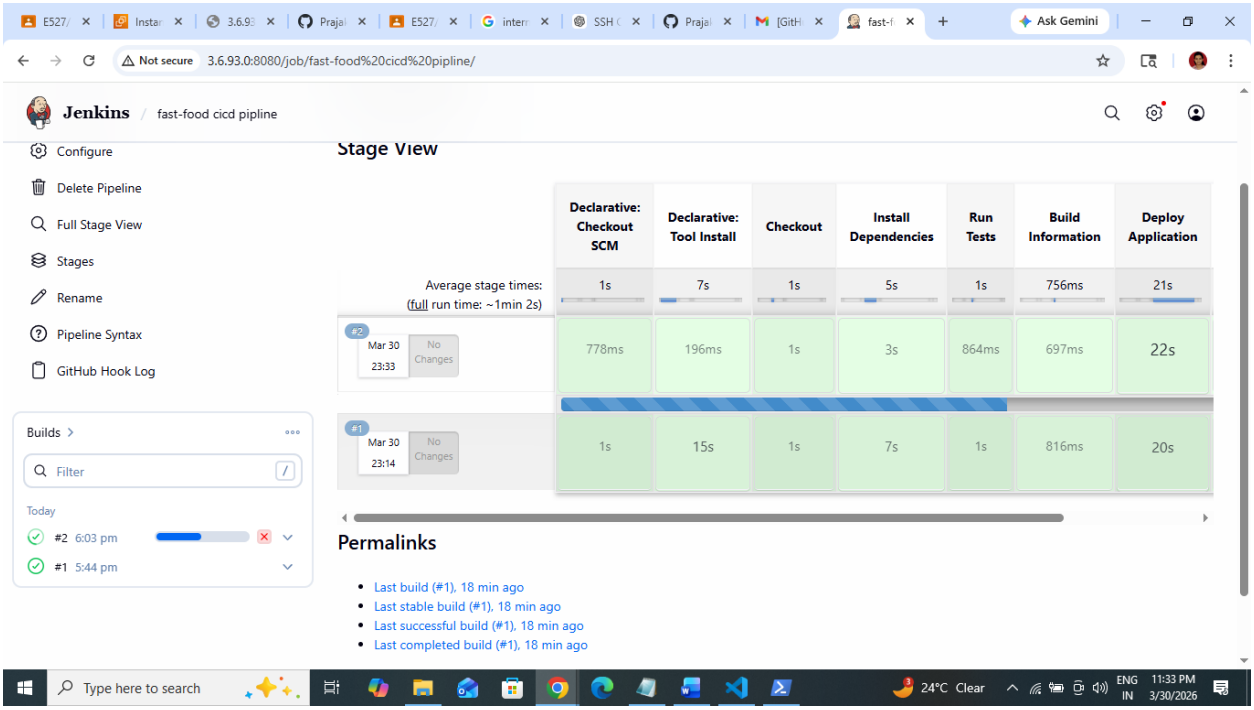
sudo usermod -aG ubuntu jenkins

2) Edit sudoers file to grant Jenkins sudo permissions (if needed)

sudo visudo

3)Restart Jenkins service to apply group membership changes

sudo systemctl restart Jenkins.



Not secure 3.6.93.0:8080/job/fast-food%20cicd%20pipline/2/console

Jenkins / fast-food cicd pipline / #2

```
user     | watching |
├────┼──────────────┼──────────┼───────┼────────┼───────┼────┼──────────┼──────────┼────────┤
────────┼────────┤
│ ⍰[1m⍰[36m0⍰[39m⍰[22m  │ fastfood-app   │ default    │ 1.0.0  │ ⍰[7m⍰[1mfork⍰[22m⍰[27m   │ 4694   │ 10s    │ 0
│ ⍰[32m⍰[1monline⍰[22m⍰[39m   │ 0%     │ 24.7mb   │ ⍰[1mjenkins⍰[22m  │ ⍰[90mdisabled⍰[39m │
└────┴──────────────┴──────────┴───────┴────────┴───────┴────┴──────────┴──────────┴────────┘
────────┴────────┘
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.541.3

Type here to search 24°C Clear ENG IN 11:43 PM 3/30/2026

ubuntu@ip-172-31-33-87: /var/www/fastfood-app

```
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Mon Mar 30 17:57:38 2026 from 223.228.137.193
ubuntu@ip-172-31-33-87:~$ ls -la /var/www/fastfood-app/
total 128
drwxr-xr-x  4 jenkins jenkins  4096 Mar 30 18:03 .
drwxr-xr-x  4 root    root     4096 Mar 30 18:03 ..
-rw-r--r--  1 jenkins jenkins 10902 Mar 30 18:03 Jenkinsfile
-rw-r--r--  1 jenkins jenkins    47 Mar 30 18:03 README.md
-rw-r--r--  1 jenkins jenkins   732 Mar 30 18:03 appspec.yml
-rw-r--r--  1 jenkins jenkins  1415 Mar 30 18:03 buildspec.yml
-rw-r--r--  1 jenkins jenkins 10814 Mar 30 18:03 index.html
drwxr-xr-x 70 jenkins jenkins  4096 Mar 30 18:03 node_modules
-rw-r--r--  1 jenkins jenkins 29362 Mar 30 18:03 package-lock.json
-rw-r--r--  1 jenkins jenkins   437 Mar 30 18:03 package.json
-rw-r--r--  1 jenkins jenkins 11860 Mar 30 18:03 script.js
drwxr-xr-x  2 jenkins jenkins  4096 Mar 30 18:03 scripts
-rw-r--r--  1 jenkins jenkins  5348 Mar 30 18:03 server.js
-rw-r--r--  1 jenkins jenkins 17494 Mar 30 18:03 style.css
ubuntu@ip-172-31-33-87:~$ cd /var/www/fastfood-app
ubuntu@ip-172-31-33-87:/var/www/fastfood-app$ pm2 start server.js --name fastfood-app
[PM2] Spawning PM2 daemon with pm2_home=/home/ubuntu/.pm2
[PM2] PM2 Successfully daemonized
[PM2] Starting /var/www/fastfood-app/server.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	mode	⍰	status	cpu	memory
0	fastfood-app	fork	0	online	0%	36.5mb

```
ubuntu@ip-172-31-33-87:/var/www/fastfood-app$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
ubuntu@ip-172-31-33-87:/var/www/fastfood-app$ pm2 list
```

id	name	mode	⍰	status	cpu	memory
0	fastfood-app	fork	0	online	0%	48.4mb

```
ubuntu@ip-172-31-33-87:/var/www/fastfood-app$
```

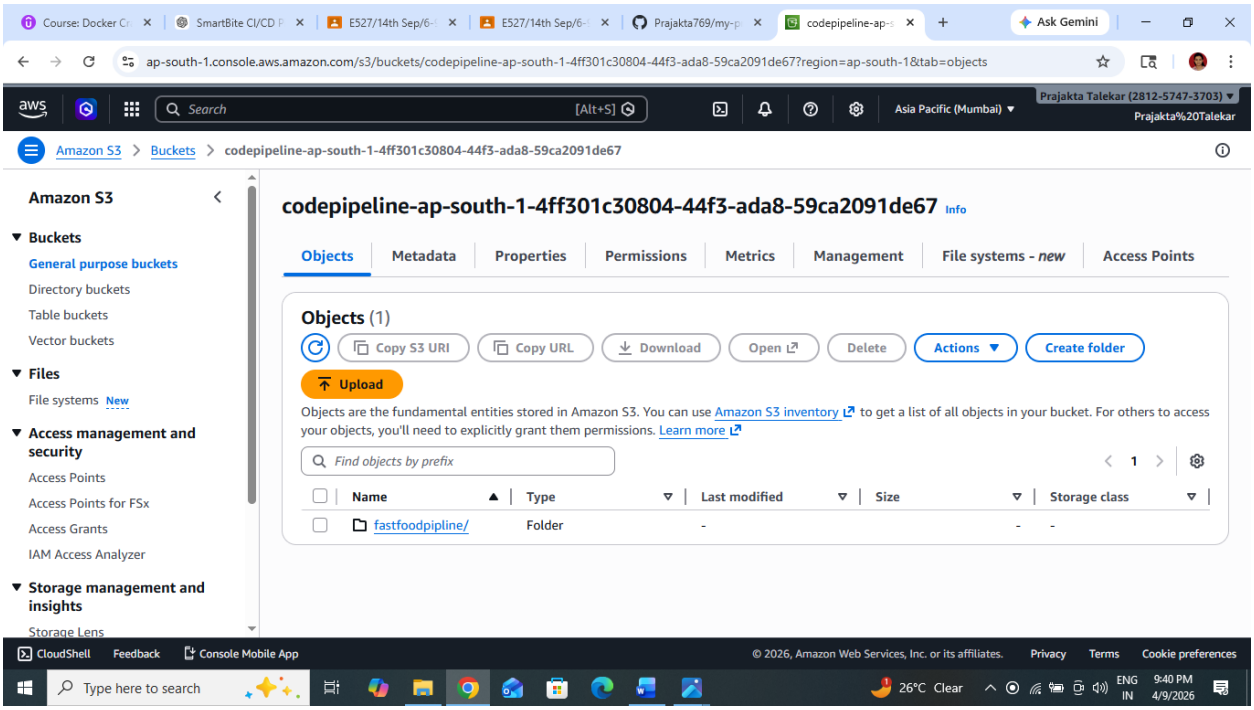
Type here to search 24°C Clear ENG IN 11:52 PM 3/30/2026

Step 8: Create S3 Bucket

Create an S3 bucket to store build artifacts.

Why?

S3 is used as a storage location for files generated during the build process.



Step 9: Create CodeBuild Project

- Configure source (GitHub or S3)
- Add buildspec.yml file

Why?

CodeBuild compiles the code and prepares it for deployment.

us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/roles/details/CodeBuild-Service-Role?section=permissions

Prajakta Talekar (2812-5747-3703)

Prajakta%20Talekar

IAM > Roles > CodeBuild-Service-Role

Policy codebuild-policy created.

CodeBuild-Service-Role Info

Allows CodeBuild to call AWS services on your behalf.

Delete

Summary

Edit

Creation date
March 31, 2026, 00:07 (UTC+05:30)

ARN
arn:aws:iam::281257473703:role/CodeBuild-Service-Role

Last activity
-

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (3) Info

You can attach up to 10 managed policies.

Simulate | Remove | Add permissions

Filter by Type

Identity and Access Management (IAM)

Search IAM

- Dashboard
- Access Management
  - User groups
  - Users
  - Roles
  - Policies
  - Identity providers
  - Account settings
  - Root access management
  - Temporary delegation requests New
- Access reports
  - Access Analyzer

CloudShell | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy | Terms | Cookie preferences

25°C Clear

ENG IN 12:28 AM 3/31/2026

fast-food-build | CodeBuild | ap

ap-south-1.console.aws.amazon.com/codesuite/codebuild/projects/fast-food-build?builds-meta=eyJmIjp7InRIeHQiOiIiLCJzdGF0dXMiOiIifSwicyI6e30sIm4iOjIwLCJplj...

Developer Tools > CodeBuild > Build projects > fast-food-build

Developer Tools
CodeBuild

- Source • CodeCommit
- Artifacts • CodeArtifact
- Build • CodeBuild
  - Getting started
  - Build projects
    - Build project
    - Settings
  - Build history
  - Report groups
  - Report history
  - Compute fleets New
  - Account metrics
- Related integrations

Project created
You have successfully created the following project: fast-food-build
Create a notification rule for this project

fast-food-build

Actions | Create trigger | Edit | Clone | Debug build | Start build with overrides | Start build

Configuration

Source provider	Primary repository	Artifacts upload location	Service role
GitHub	Prajakta769/my-project-2	fast-foodcicd	arn:aws:iam::281257473703:role/CodeBuild-Service-Role

Public builds
Disabled

Build history | Batch history | Project details | Build triggers | Metrics | Debug sessions

Build history

Stop build | View artifacts | View logs | Delete builds | Retry build | Debug build

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/iam/home?region=ap-south-1#/roles/details/CodeBuild-Service-Role?section=permissions

IAM > Roles > CodeBuild-Service-Role

Identity and Access Management (IAM)

- Dashboard
- Access Management
  - User groups
  - Users
  - Roles
  - Policies
  - Identity providers
  - Account settings
  - Root access management
  - Temporary delegation requests New
- Access reports
  - Access Analyzer

Policy codebuild-policy created.

CodeBuild-Service-Role Info

Allows CodeBuild to call AWS services on your behalf.

Delete

Summary

Edit

Creation date	ARN
March 31, 2026, 00:07 (UTC+05:30)	arn:aws:iam::281257473703:role/CodeBuild-Service-Role
Last activity	**Maximum session duration**
-	1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

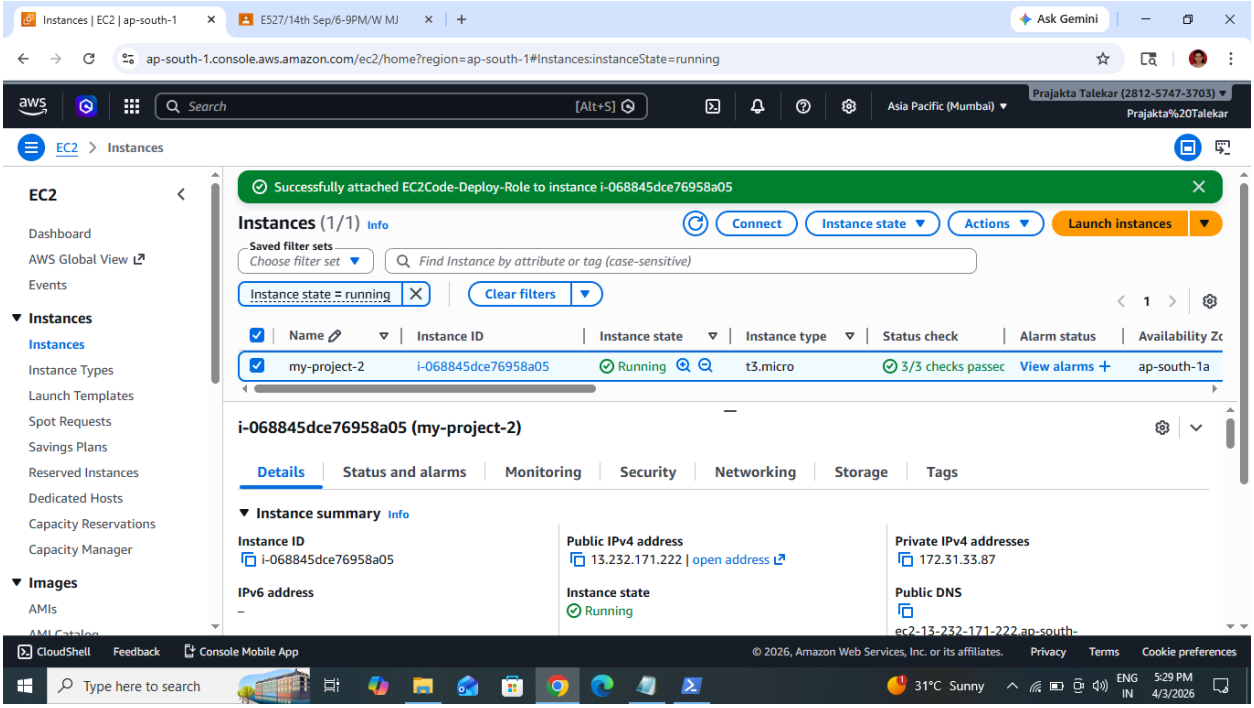
Permissions policies (3) Info

You can attach up to 10 managed policies.

Simulate | Remove | Add permissions

Filter by Type

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

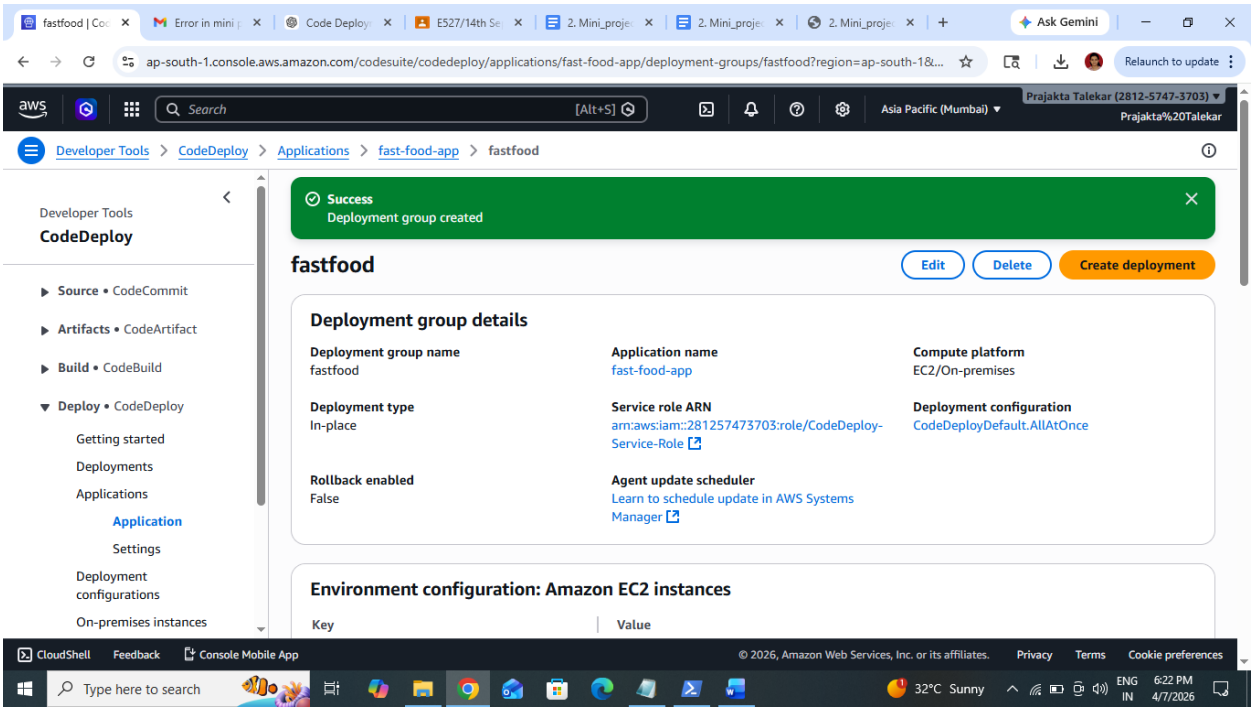


Step 10: Create CodeDeploy Application

- Create a CodeDeploy application
- Create a deployment group
- Attach EC2 instance

Why?

CodeDeploy automates deployment of the application to the server.



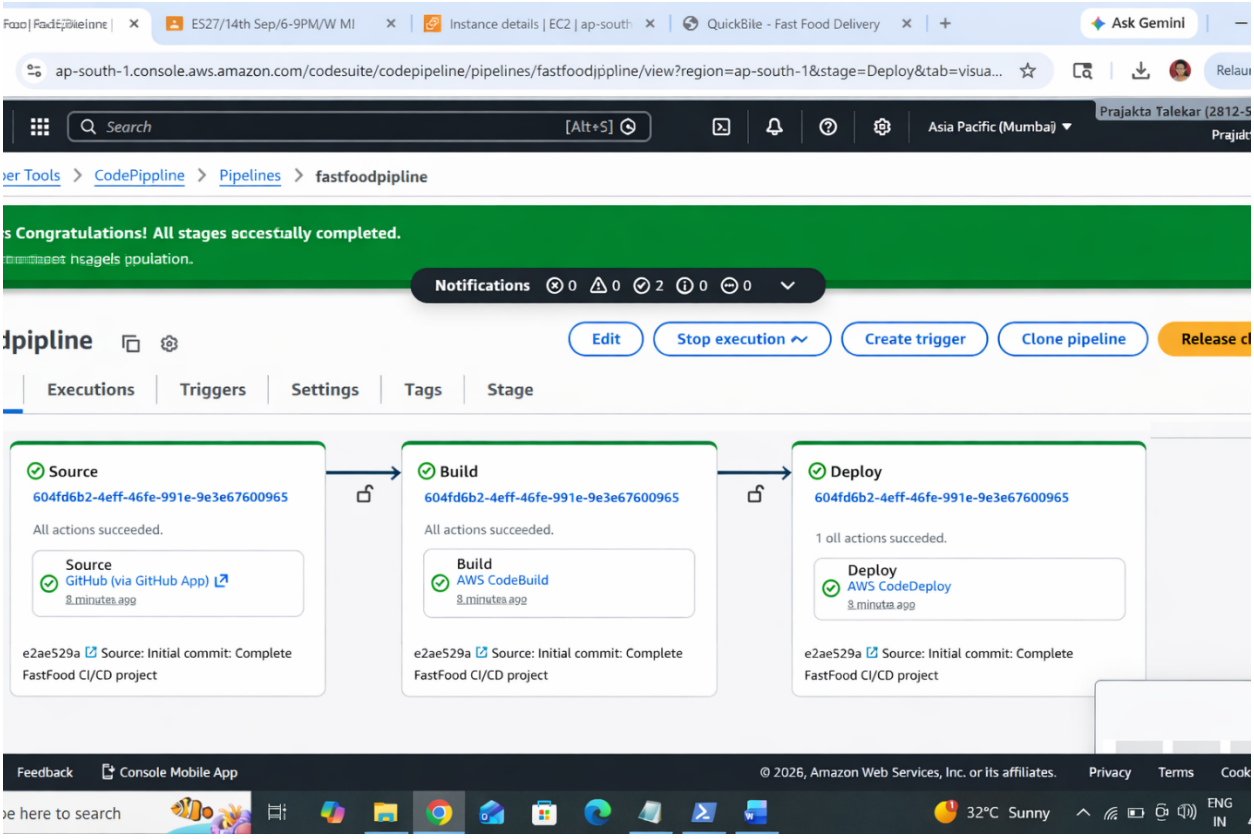
Step 11: Create CodePipeline

Create a pipeline with the following stages:

1. Source (GitHub/S3)
2. Build (CodeBuild)
3. Deploy (CodeDeploy)

Why?

CodePipeline automates the entire CI/CD workflow.



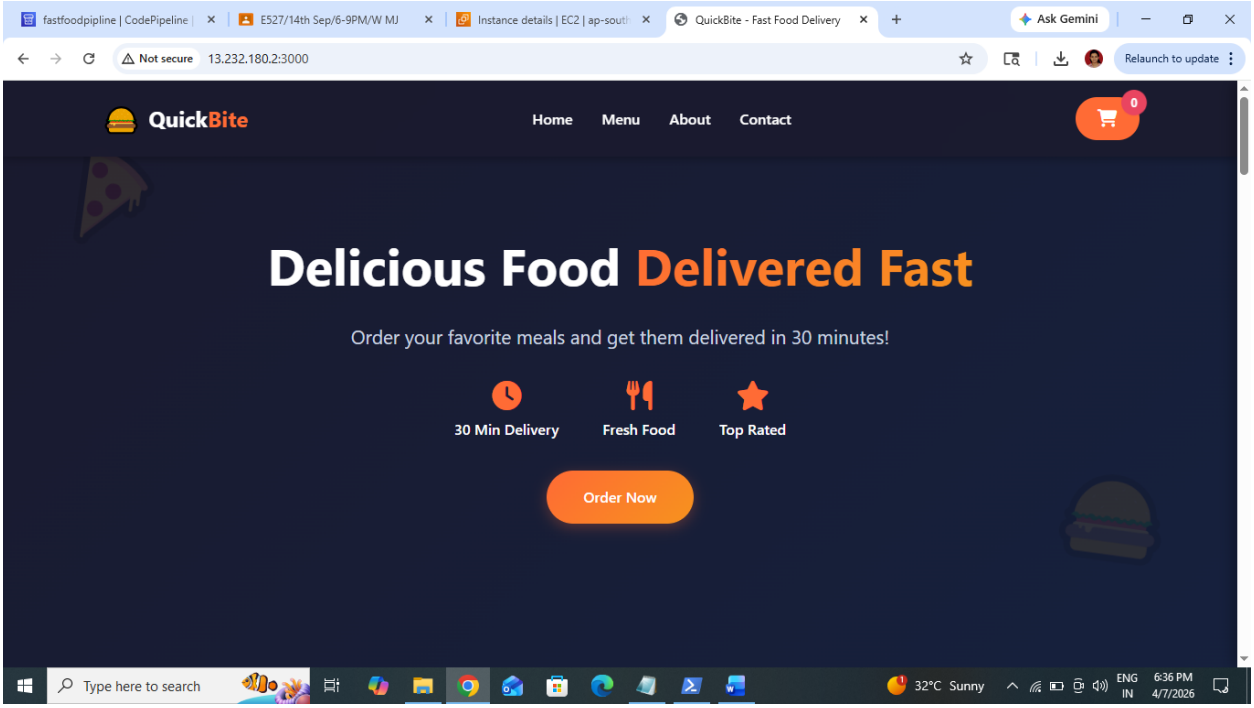
Step 12: Deploy Application

Once the pipeline runs successfully:

- Application gets deployed to EC2
- Access it using public IP

Why?

This confirms that the CI/CD pipeline is working correctly.



Conclusion

This project demonstrates how a Fast Food application can be deployed using a **fully automated CI/CD pipeline**. By integrating Jenkins with AWS services like CodePipeline, CodeBuild, and CodeDeploy, we achieved:

- Faster deployment
- Reduced manual work
- Continuous integration and delivery
- Improved reliability and scalability

This setup is highly useful in real-world DevOps environments where frequent updates and quick deployments are required.